



Unidade 4 — Por que desenvolver seu próprio Framework?



Objetivos de Aprendizagem

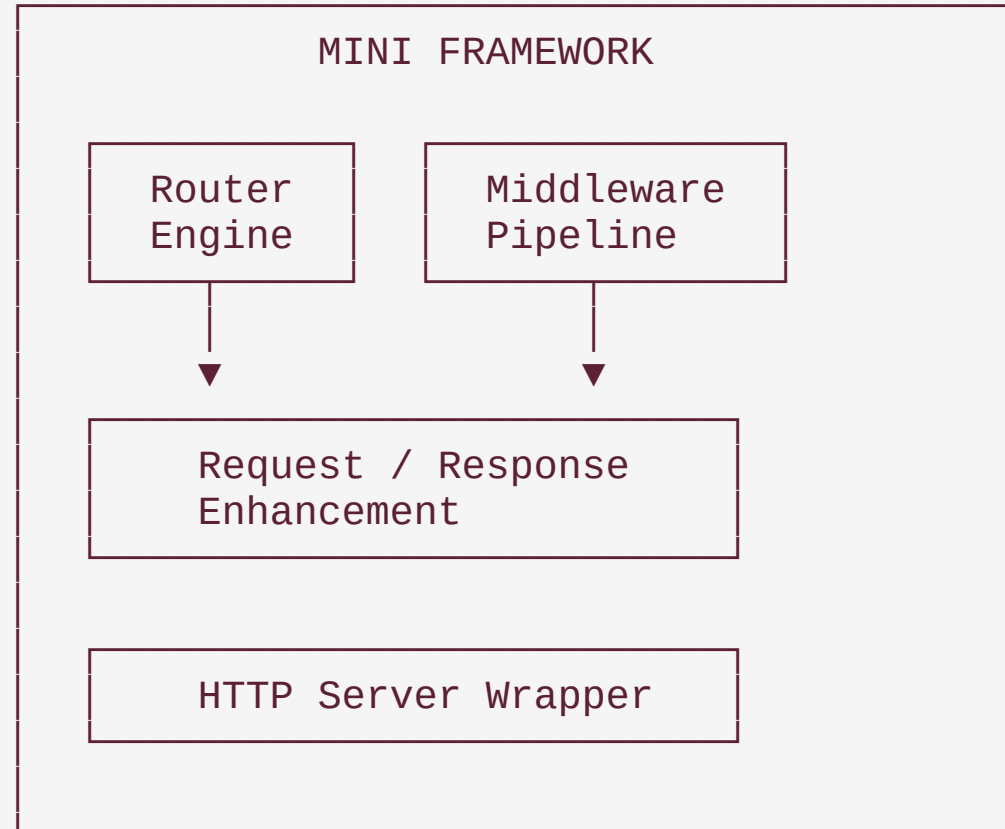
Ao final desta aula, o estudante deverá ser capaz de:

- **Projetar** a arquitetura de um mini framework;
- **Identificar** os componentes necessários;
- **Definir** a API pública do framework;
- **Analisar** como o Express.js resolve os mesmos problemas.

Taxonomia de Bloom: Projetar, Identificar, Definir, Analisar.



Componentes do Mini Framework





API Pública Desejada

```
// Criar aplicação
const app = MiniFramework();

// Middleware global
app.use(json());

// Rota GET
app.get('/api/usuarios', (req, res) => {
  res.json(usuarios);
});

// Rota com parâmetro
app.get('/api/usuarios/:id', (req, res) => {
  const usuario = usuarios.find(u => u.id == req.params.id);
  res.json(usuario);
});

// Rota POST
app.post('/api/usuarios', (req, res) => {
  usuarios.push(req.body);
  res.status(201).json(req.body);
});

// Iniciar servidor
app.listen(3000, () => console.log('Rodando na porta 3000'));
```





Fluxo de uma Requisição no Framework

1. HTTP recebe requisição
- ↓
2. Enhance req e res (adicionar métodos)
- ↓
3. Executar middlewares (pipeline)
- ↓
4. Router encontra handler
- ↓
5. Handler executa (controller)
- ↓
6. Resposta é enviada



Síntese da Aula

Conceitos principais:

1. O mini framework terá quatro componentes: **Router**, **Middleware**, **Request/Response Enhancement**, **HTTP Wrapper**;
2. A API pública será semelhante ao **Express.js**;
3. O fluxo é: HTTP → Enhance → Middlewares → Router → Handler → Response;
4. O design deve vir **antes** do código;
5. A semelhança com o Express é **intencional** para facilitar o aprendizado.

Pergunta reflexiva:

Se nosso mini framework tem a mesma API que o Express, qual é a diferença?

Próxima aula: Construindo um Router — Vamos implementar o componente de roteamento.

